

```
int ioctl(struct inode *i, struct file *f, unsigned int cmd,
unsigned long arg);
```

However, from kernel 2.6.35, it changed to:

```
long ioctl(struct file *f, unsigned int cmd, unsigned long arg);
```

If there is a need for more arguments, all of them are put in a structure, and a pointer to the structure becomes the 'one' command argument. Whether integer or pointer, the argument is taken as a long integer in kernel-space, and accordingly type-cast and processed.

ioctl() is typically implemented as part of the corresponding driver, and then an appropriate function pointer is initialised with it, exactly as with other system calls like *open()*, *read()*, etc. For example, in character drivers, it is the *ioctl* or *unlocked_ioctl* (since kernel 2.6.35) function pointer field in the *struct file_operations* that is to be initialised.

Again, like other system calls, it can be equivalently invoked from user-space using the *ioctl()* system call, prototyped in `<sys/ioctl.h>` as:

```
int ioctl(int fd, int cmd, ...);
```

Here, *cmd* is the same as what is implemented in the driver's *ioctl()*, and the variable argument construct (...) is a hack to be able to pass any type of argument (though only one) to the driver's *ioctl()*. Other parameters will be ignored.

Note that both the command and command argument type definitions need to be shared across the driver (in kernel-space) and the application (in user-space). Thus, these definitions are commonly put into header files for each space.

Querying driver-internal variables

To better understand the boring theory explained above, here's the code set for the 'debugging a driver' example mentioned earlier. This driver has three static global variables: *status*, *dignity*, and *ego*, which need to be queried and possibly operated from an application. The header file *query_ioctl.h* defines the corresponding commands and command argument type. A listing follows:

```
#ifndef QUERY_IOCTL_H
#define QUERY_IOCTL_H

#include <linux/ioctl.h>

typedef struct
{
    int status, dignity, ego;
} query_arg_t;
```

```
#define QUERY_GET_VARIABLES_IOR('q', 1, query_arg_t *)
#define QUERY_CLR_VARIABLES_IO('q', 2)

#endif
```

Using these, the driver's *ioctl()* implementation in *query_ioctl.c* would be as follows:

```
static int status = 1, dignity = 3, ego = 5;

#if (LINUX_VERSION_CODE < KERNEL_VERSION(2,6,35))
static int my_ioctl(struct inode *i, struct file *f, unsigned int
cmd, unsigned long arg)
#else
static long my_ioctl(struct file *f, unsigned int cmd, unsigned
long arg)
#endif
{
    query_arg_t q;

    switch (cmd)
    {
        case QUERY_GET_VARIABLES:
            q.status = status;
```



**Achieve your
CAREER
GOAL**

with World
Class Certifications

Enroll for RH 318
(Virtualization) and avoid
Windows-2008 free!

- RHCSA
- RHCE
- RHCSS
- RHCA
- RHCDS
- RHCVA
- SUSE
- SOLARIS
- VERITAS

- Best Certified Red Hat Training Partner, South India for 3 times in row
- Second in South India in terms of number of RHCEs
- RHCE of the year-09, bagged by a COSS student
- 500+ RHCEs Every year
- 16 RHCSA so far, the highest in India
- NOVELL Gold Training Partner and Novell Gold Solution Provider
- Consistent 97% passing rate in RHCE examination
- Undisputed leader in South India for Open Source Training

Save Rupees 5000/- on
RHCE Examination
Limited Period Offer

Complete Open Source Solutions

HYDERABAD • SECUNDERABAD • DILSUKHINAGAR

Hyderabad: #512, Aditya Trade Center, Ammerpet, Hyderabad-58 Tel: +91-40-66773365.
Fax: +91-40-66463365. Email: contact@coSSIDa.net web: http://www.coSSIDa.net